QUEUEDESK · SYNTHETIC PAYMENT LEARNING LAB

Durable payments, learned by breaking them.

Send a payment once — even if you click twice. Lose a message — nothing is lost. Smell fraud — it gets held for review.

`local-only` `177 backend tests` `74 dashboard tests` `SC01–SC12 green` `Jev live · $1 cap`

WHY IT EXISTS

A lab, not a demo

Real payment systems fail in boring, expensive ways: a message gets lost, a customer is charged twice, fraud slips through quietly. QueueDesk is a **working miniature** of such a system where you can trigger each failure on demand and watch the safety nets catch it.

- **Real parts, toy money:** an actual database, an actual message queue — but every dollar is pretend
- **Honest status:** if something is unresolved, the screen says “pending” — it never fakes “all good”
- **12 disaster drills:** unplug the queue, crash mid-payment, send poison — each with a saved report card
- **A live AI guard:** a real fraud-checking AI screens payments under a strict $1 spending cap

THE FLOW

Journey of a payment

Follow $125 from “send” to “settled”. Seven steps — each one either safely stored or visibly waiting.

1. **Front door**
   The app receives your $125. Sent twice by accident? Counted once — the receipt key proves it.
2. **Written down**
   Payment + delivery note are saved together. A crash here loses nothing — both or neither.
3. **Conveyor belt**
   A message queue carries the payment onward. Bad messages are set aside, never jam the belt.
4. **Sorter**
   A worker picks it up, ignores duplicates, and looks up your history before anything else.
5. **Security check**
   Rules + AI judge it: pass, hold for review, block, or “couldn’t check — hold safely”.
6. **Cashier**
   A pretend bank settles it: paid, declined, or “unsure”. Lost answers are chased up later.
7. **Receipt filed**
   The final result + full history are stored, and the dashboard shows every step.

ARCHITECTURE

Four parts, one laptop

Everything runs on this machine — nothing in the cloud except the AI fraud-checker.

1 · The engine (takes payments)

A web service receives payments and hands them to the queue

The database remembers everything: payments, delivery notes, fraud checks, history

The message queue delivers each payment at least once; the worker ignores repeats

2 · The dashboard (watch only)

A web UI shows payments, fraud holds, queue depths, and traces

Strictly read-only — looking can never change or break anything

Unknowns show as dashes; “pending” means pending, never guessed

3 · The security desk

Simple rules plus an AI adviser screen every payment **before** money moves

Every decision and its reasons are stored for review

4 · The drill instructors

A traffic generator sends scripted payments — normal ones and fraud attempts

A verifier runs 12 disaster drills and grades each with a report card

RISK GATE

Rules decide. AI advises.

Three simple tripwires catch the obvious stuff; an AI adviser double-checks. And a strict rule: **the AI can never approve, block, or accuse anyone** — it only answers two yes/no questions as background.

The three tripwires

Stranger danger — first-ever payment to this person? → hold for review

Weird amount — $87,000 when you usually send $500? → hold for review

Hard limit — $95,000+ to a stranger? → blocked

If the AI can't be reached, rules still apply; quiet payments → held safely

Four strictness settings

Off — everything sails through (for testing the plumbing)

Rules only — tripwires without the AI

Rules + pretend AI — a free built-in stand-in, same questions

Rules + real AI — the live model, under a $1 spending cap

LIVE AI · $1 CAP

The AI guard, on a prepaid meter

"Jev" is an outside AI service that answers two questions per payment: *is the recipient new?* and *is the amount unusual?* It sees only numbers — never names, never account details.

- **Prepaid $1 meter:** every answer costs a fraction of a cent; the meter lives in the database, so restarting the app can't reset it
- **Hard stop at $1:** once the dollar is spent, checks pause and payments queue for review — shown on screen, never silent
- **Observed cost:** one real check took 0.3 seconds and cost $0.00002 — two-thousandths of a cent
- **It already taught us something:** the AI reports "confidence" differently than our test double did — found on the first live call, fixed and locked in with a test

FAILURE LAB

Twelve disaster drills — on purpose

Each drill breaks one thing, then checks the safety net held. A few favorites:

“Unplug it mid-flight”

Kill the message queue — payments pile up safely, drain cleanly after restart

Crash halfway through saving — redelivery produces exactly one result, never two

Send a poison message — quarantined with a reason; everything else keeps flowing

“Confuse it”

Click “pay” twice — charged once; **change the amount mid-retry** — rejected loudly

Lose the bank's reply — chased up and settled; truly unknown stays “pending”

Flood it / starve it — bursts absorb, deadlines admit only what fits

One command runs every drill plus all unit tests and grades the report card: pass, or exactly what failed.

DASHBOARD

Read-only operations view

A control tower for the lab: six screens, every number traceable to a real reading.
Nothing here can change anything — it's windows, not levers.

The three you'll use most

Overview — the big picture: how many paid, how many held, queue health

Payments — the ledger, newest first; filter by experiment run

Risk review — the held-payments queue with reasons, plus the AI spend meter

For digging deeper

Queues / Ops — is the conveyor belt backed up? any guards tripped?

Telemetry / Scenarios — follow one payment's trail; replay past drills

Missing data shows as a dash — the screen would rather admit ignorance than invent a number

STACK

What's inside (the stack)

Industry-standard parts with pinned versions — the same setup rebuilds bit-for-bit on any machine.

PART	TECHNOLOGY	WHAT IT DOES, PLAINLY
The engine	Java 25 · Spring Boot 4.0.8	The web service that takes payments and runs the whole flow
The dashboard	Angular 22 · Tailwind 4.3	The control-tower screens you look at (built to match the designs)
The memory	PostgreSQL 17	The database that forgets nothing: payments, checks, history, AI meter
The conveyor	IBM MQ	The industrial message queue that never drops a payment
The boxes	Kubernetes (kind)	Runs each part in its own container, all on this laptop
The AI guard	TypeSafe Jev	The outside fraud-checker, prepaid $1, numbers-only — no names
The flight recorder	OpenTelemetry · logs	Every payment leaves a trace you can follow step by step

OPERATE

Try it: four commands

1 · Start everything

```
./start-lab
```

Launches the database, queue, engine, and dashboard. Safe to re-run; won't fight for busy ports.

2 · Send 20 payments, fraud mixed in

```
./scripts/generate --run-id w1 --count 20 --scenario risk-2 --send --observe
```

Shows the plan first without sending; --send fires it, --observe watches each one settle.

3 · Clean slate · 4 · Prove health

```
./scripts/reset-lab  &  ./scripts/verify
```

Wipe the pretend data between experiments; run every test and all 12 disaster drills.

RECEIPTS

Verified, not claimed

177
backend tests green

74
dashboard specs green

12
failure scenarios green

$0.00006
total cloud spend of $1

- **Fraud mix caught live:** 20 payments in, 6 held with reasons — 5 giant amounts, 1 first-timer
- **AI check proved live:** a real fraud screen answered in 0.3 seconds for $0.00002
- **Screens match the designs:** every page screenshot-checked against its mockup

NEXT

Where it goes from here

- **Grade the guards:** run rules and AI side by side on the same fraud — who catches more, who cries wolf?
- **Speed vs cost:** how slow and pricey does the AI get at high volume?
- **Sneakier fraud:** rapid-fire bursts, split-up amounts, fraud rings
- **Let reviewers act:** today the panel is look-only — approve/release buttons come later
- **Tune the tripwires:** set the real limits using measured data, not guesses

QUEUEDESK

Break it on purpose. Learn for real.

Local-only · honest states · proved failure handling · live AI under a hard cap

`./start-lab` `:8080 API` `:4200 dashboard`